SQL Injection Vulnerability Allowing Login Bypass

What is SQL Injection?

SQL Injection (SQLi) is a web security vulnerability that occurs when an application improperly validates or sanitizes user input before including it in an SQL query. An attacker can manipulate the input to alter the intended SQL query, allowing unauthorized access to data, modification of database contents, authentication bypass, or execution of administrative operations on the database.

In a login bypass attack, SQL injection is used to modify the authentication query so that the application grants access without requiring valid credentials. This happens when user-supplied input is interpreted as part of the SQL command rather than as data.

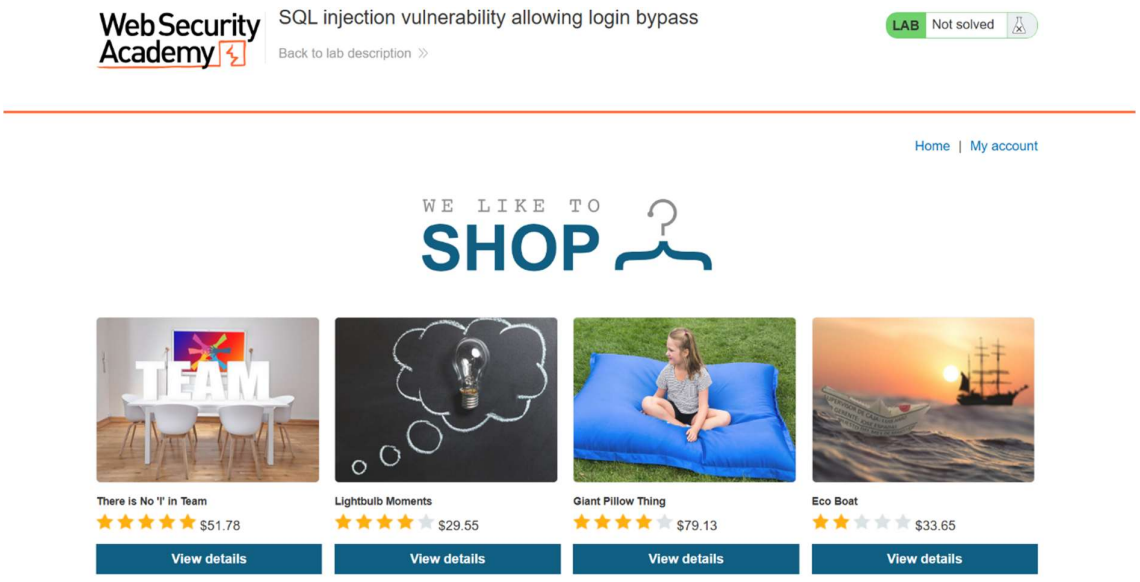
SQL injection remains one of the most critical web application vulnerabilities and can lead to data breaches, privilege escalation, and complete compromise of the underlying database system.

For example, a vulnerable login query might look like this:
SELECT * FROM users WHERE username = 'USER_INPUT' AND password = 'PASSWORD_INPUT'

If an attacker inputs ' OR 1=1 -- into the username field, the resulting executed query becomes:
SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = '...'

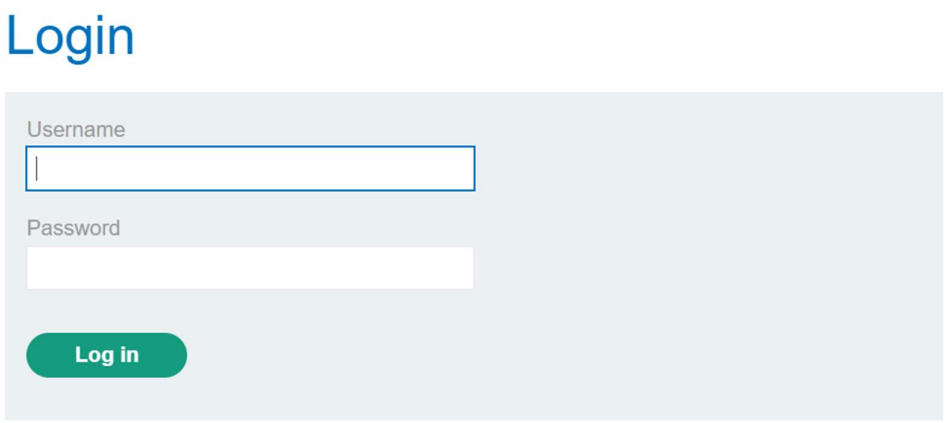
Because 1=1 is always true, and the double dash (--) comments out the rest of the query, the database returns the first user in the table, bypassing authentication entirely. [1, 2]

After accessing Lab



This is the website which we will be working on

After clicking **My Account** it will ask for username and password as we don't know both of them we will try to bypass it using SQL Injection



This is our login page

We will try to enter random username and use **BurpSuite** intercept the traffic

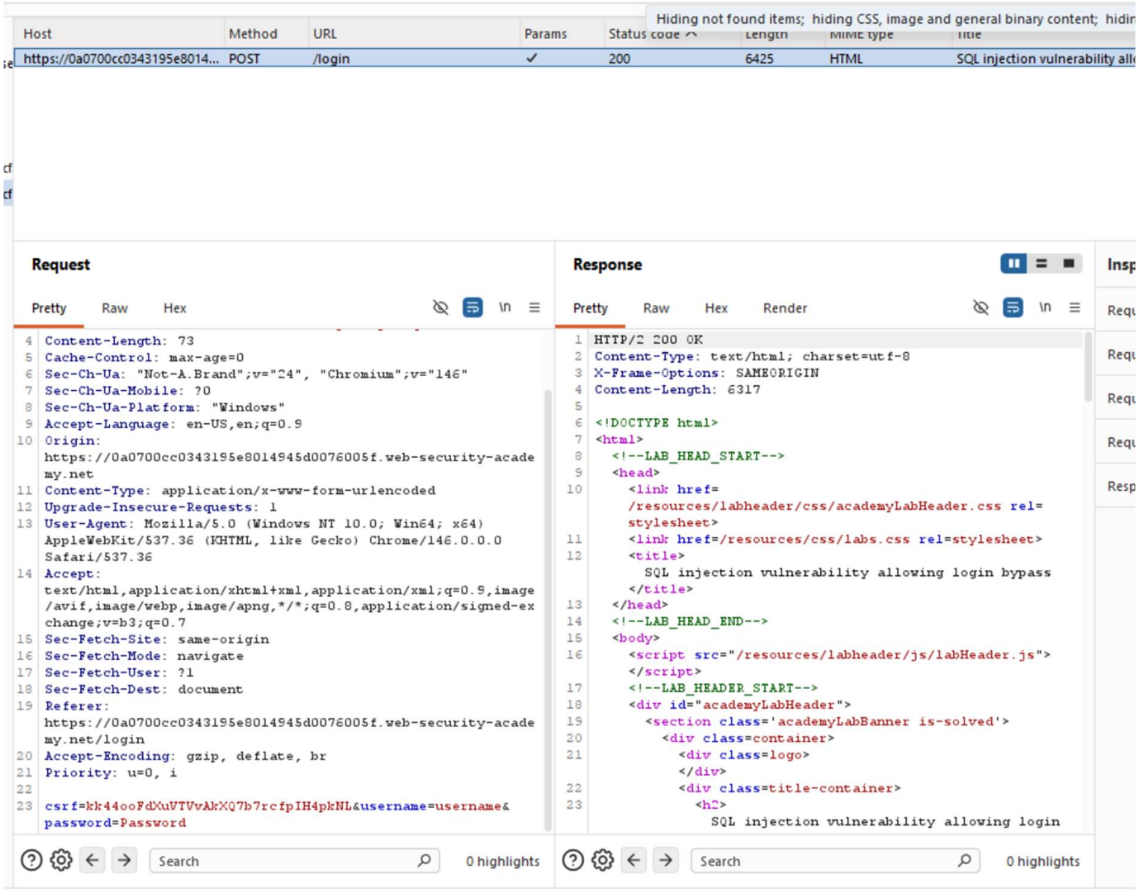
For now I entered

{

username:"username"

password:"Password"

}



After intercepting the traffic, in the request side I got my username and password which I entered

And after sending it to the repeater I changed the username to administrator'-- and entered the password as it is then I was able to login and bypassed the login.

And finally the lab was solved

The key takeaway's

1)SQL Injection occurs when user input is incorporated into SQL queries without proper validation or sanitization.

2)Authentication mechanisms can be bypassed if user-supplied input is interpreted as SQL code instead of data.

3)Burp Suite is a useful tool for intercepting, analyzing, and modifying HTTP requests during web application security testing.

4)Login forms are common targets for SQL Injection attacks because they directly interact with backend databases.

5)Developers should use parameterized queries (prepared statements) instead of dynamically building SQL queries with user input.

6)Input validation and output encoding are essential security controls that help prevent SQL Injection vulnerabilities.

7)Principle of Least Privilege should be applied to database accounts to minimize the impact of a successful SQL Injection attack.

8)Security testing should be performed regularly to identify and remediate vulnerabilities before deployment.

9)Even a simple SQL Injection vulnerability can lead to unauthorized access, sensitive data exposure, and complete application compromise.

10)Understanding how SQL Injection works helps security professionals identify, exploit (in authorized environments), and mitigate such vulnerabilities effectively.